

LeetCode 2065: Maximum Path Quality of a Graph

Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 26, 2026

Target Companies: SAP, Atlassian

Problem Understanding & Key Points

- **Graph Properties:** The graph is *undirected*. Each edge has u , v , and $time$. Each node has a specific value.
- **Objective:** Start from node 0 and end at node 0 . Return the path with the maximum quality (sum of values of nodes).
- **Constraint:** The total time of the path must be under $maxTime$.

Why not Dijkstra's Algorithm?

Whenever we visit a node for the first time, we collect its value. If we visit some node again, we don't collect its value again.

If we use standard Dijkstra's, we may go to the best city and return using the best path, but that won't be optimal as you are not collecting values on the return path.

State Explosion in Dijkstra's

The true Dijkstra's state would be $\{node, time, visitedset\}$ where the *visited set* stores which nodes have already contributed their values. This requires 2^N **masks**, which completely destroys efficiency.

Here, the constraints are small, so the optimal approach is **DFS + Backtracking**.

Crucial Detail

DON'T STOP whenever you reach node == 0, because there might still be time left to collect more value and return again.

Approach 1: BFS with State (Queue)

```
class Solution {
public:
    int maximalPathQuality(vector<int>& values, vector<vector<int>>& edges, int
maxTime) {

        int n = values.size();

        // {nextnode, time}
        vector<vector<pair<int,int>>> adj(n);

        for(auto &it : edges) {
            int u = it[0];
            int v = it[1];
            int t = it[2];

            adj[u].push_back({v, t});
            adj[v].push_back({u, t});
        }

        // {value, {node, time}, seen}
        queue<pair<pair<int, pair<int, int>>, vector<int>>> q;
        vector<int> seen(n, 0);

        seen[0] = 1;
        // {value, {node, time}, seen}
        q.push({values[0], {0, 0}, seen});

        int ans = 0;
        while(!q.empty()) {

            int currvalue = q.front().first.first;
            int currnode = q.front().first.second.first;
            int currtime = q.front().first.second.second;
            vector<int> currseen = q.front().second;
            q.pop();
```

```

        if(currnode == 0) {
            ans = max(ans, currvalue);
        }

        for(auto &it : adj[currnode]) {

            int nextnode = it.first;
            int edgetime = it.second;
            int newtime = currtime + edgetime;

            if(newtime > maxTime) {
                continue;
            }

            int newvalue = currvalue;
            vector<int> newseen = currseen;

            // agarr pahli dafa dekha he to seen krke uski value daal do
            if(newseen[nextnode] == 0) {
                newseen[nextnode] = 1;
                newvalue += values[nextnode];
            }

            q.push({newvalue, {nextnode, newtime}, newseen});
        }
    }

    return ans;
}
};

```

Approach 2: DFS + Backtracking (Optimal)

```

class Solution {
public:
    int ans = 0;

    void dfs(int currnode, int currtime, int currval, vector<int>& seen,
vector<vector<pair<int,int>>>& adj, vector<int>& values, int maxtime) {

        if(currnode == 0) {
            ans = max(ans, currval);
            // wont stop as ho skta aur time baccha ho kahi aur jaake wapas aane
            ke liue
        }
    }
}

```

```

        for(auto &it : adj[currnode]) {
            int newnode = it.first;
            int cost = it.second;
            int newtime = currtime + cost;
            int newval = currval;

            if(newtime > maxtime) continue;

            if(seen[newnode] == 0) {
                seen[newnode] = 1;
                newval += values[newnode];
                dfs(newnode, newtime, newval, seen, adj, values, maxtime);

                // since seen is changed we need to backtrack to explore other
paths as well
                seen[newnode] = 0;
            }
            else {
                // koi change nhi to simple dfs no backtrack
                dfs(newnode, newtime, newval, seen, adj, values, maxtime);
            }
        }
    }

    int maximalPathQuality(vector<int>& values, vector<vector<int>>& edges, int
maxTime) {

        int n = values.size();

        // {nextnode, time}
        vector<vector<pair<int,int>>> adj(n);

        for(auto &it : edges) {
            int u = it[0];
            int v = it[1];
            int t = it[2];

            adj[u].push_back({v, t});
            adj[v].push_back({u, t});
        }

        vector<int> seen(n, 0);
        seen[0] = 1;

        // we will do dfs and backtrack
        // dfs function will have
currnode,currtime,currval,seen,adj,values,maxtime
        dfs(0, 0, values[0], seen, adj, values, maxTime);
    }

```

```
        return ans;  
    }  
};
```

© 2026 Arpit Raj, LNMIIT Jaipur. All rights reserved.